

Lập trình các thuật toán lập lịch CPU

Đỗ Hoàng Gia Bảo - 23020783

Khoa Điện tử - Viễn Thông

Ngày 23 tháng 3 năm 2026

Tóm tắt nội dung

Báo cáo sẽ trình bày các bước lập trình các thuật toán lập lịch CPU: FCFS, SJF (cho phép dừng, không cho phép dừng), Lập lịch ưu tiên (cho phép dừng, không cho phép dừng), Round Robin, Hàng đợi đa cấp, Hàng đợi đa cấp phản hồi

Mục lục

1	Giải thích các thuật toán lập lịch CPU trong chương trình	3
1.1	Cấu trúc dữ liệu và Lập lịch không cho phép dừng (Non-Preemptive)	3
1.2	Lập lịch cho phép dừng (Preemptive)	3
1.3	Round Robin và Hàng đợi đa cấp (MLFQ)	4
1.4	Tính toán hiệu năng	4

1 Giải thích các thuật toán lập lịch CPU trong chương trình

Chương trình sử dụng ngôn ngữ Python với thư viện Tkinter để mô phỏng các thuật toán lập lịch. Báo cáo sẽ trình bày từng thuật toán được cài đặt:

1.1 Cấu trúc dữ liệu và Lập lịch không cho phép dừng (Non-Preemptive)

Mỗi tiến trình được quản lý bởi lớp `Process` lưu trữ các thông số: *PID*, *Arrival Time*, *Burst Time*, *Priority* và các biến trạng thái như `remaining_time`.

- **First-Come, First-Served (FCFS):** Đây là thuật toán đơn giản nhất. Mã nguồn thực hiện sắp xếp danh sách tiến trình theo `arrival_time`. CPU phục vụ tiến trình nào đến trước cho đến khi kết thúc (`curr + p.burst`).
- **Shortest Job First (SJF - Non-Pre):** Tại mỗi thời điểm CPU rảnh, chương trình tìm trong danh sách các tiến trình đã đến (`p.arrival <= curr`) tiến trình có `burst_time` nhỏ nhất để thực thi. Do là không cho phép dừng, tiến trình sẽ chạy trọn vẹn thời gian thực hiện.
- **Priority (Non-Pre):** Tương tự như SJF, nhưng thay vì chọn theo thời gian thực hiện, CPU chọn tiến trình có chỉ số `priority` nhỏ nhất (ưu tiên cao nhất).

1.2 Lập lịch cho phép dừng (Preemptive)

Trong các thuật toán này, CPU liên tục kiểm tra lại danh sách tiến trình mỗi khi có một đơn vị thời gian trôi qua hoặc có tiến trình mới đến.

- **Shortest Remaining Time First (SRTF/SJF Preemptive):** Hàm `algo_sjf_preemptive` thực hiện vòng lặp `while completed < n`. Tại mỗi đơn vị thời gian (`curr_time`), nó chọn tiến trình có `remaining_time` thấp nhất. Nếu một tiến trình mới đến có thời gian còn lại ngắn hơn tiến trình đang chạy, CPU sẽ chuyển mạch (context switch).
- **Priority (Preemptive):** Hàm `algo_priority_preemptive` hoạt động tương tự SRTF nhưng tiêu chí lựa chọn là `min(available, key=lambda x: x.priority)`.

1.3 Round Robin và Hàng đợi đa cấp (MLFQ)

Đây là các thuật toán phức tạp hơn, sử dụng khái niệm *Time Quantum* (Định mức thời gian).

- **Round Robin (RR):** Trong mã nguồn, RR được xử lý như một trường hợp đặc biệt của MLFQ với một mức hàng đợi duy nhất và một giá trị q (Quantum). Mỗi tiến trình chỉ được chạy tối đa q thời gian, sau đó phải xuống cuối hàng đợi để nhường CPU.
- **Multilevel Feedback Queue (MLFQ):** Đây là thuật toán linh hoạt nhất được cài đặt trong hàm `algo_mlfq`. Chương trình sử dụng 3 hàng đợi (`queues[0, 1, 2]`):
 1. **Q0 (Cấp 1):** Sử dụng Round Robin với Quantum $q_1 = 4$. Nếu tiến trình không kết thúc, nó bị đẩy xuống Q1.
 2. **Q1 (Cấp 2):** Sử dụng Round Robin với Quantum $q_2 = 8$. Nếu vẫn chưa xong, nó bị đẩy xuống Q2.
 3. **Q2 (Cấp 3):** Sử dụng FCFS để xử lý nốt các tiến trình dài (CPU-bound).

Lưu ý: Bất cứ khi nào có tiến trình mới đến Q0, nó sẽ có quyền ưu tiên cao hơn các hàng đợi bên dưới.

1.4 Tính toán hiệu năng

Sau khi mô phỏng xong, chương trình tính toán các chỉ số quan trọng để đánh giá thuật toán:

$$Turnaround\ Time = Completion\ Time - Arrival\ Time \quad (1)$$

$$Waiting\ Time = Turnaround\ Time - Burst\ Time \quad (2)$$

Các kết quả này được hiển thị trực quan qua biểu đồ Gantt trên thành phần **Canvas** và bảng thống kê **Treeview**.